RV College of Engineering®

Go, change the world

Unit – III

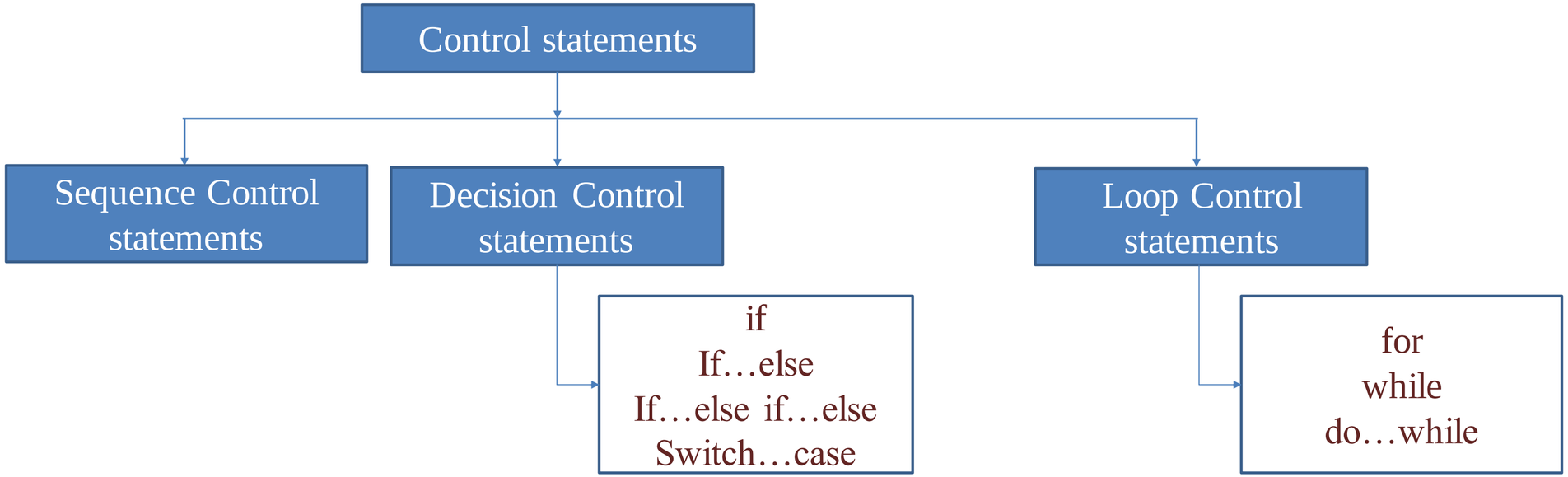
Decision Control and Looping Statements, and Arrays

Contents

1. Introduction to Decision Control Statements
2. Conditional Branching Statements
   i. if Statement
   ii. if–else Statement
   iii. if–else–If Statement
   iv. switch-case
3. Iterative statements
   i. while
   ii. do-while
   iii. for
4. Loop control statements
5. Arrays
   i. Declaration of Arrays.
   ii. Accessing elements and Storing values in an array.
   iii. Operations on Arrays- Traversing, Inserting and Deletion of elements in an array.
   iv. Operations on Two Dimensional arrays.

1. Control statements

- Control statements are those, which exhibits controlled execution of statement/statements block in the program such as decision, looping as well as loop regulating statements.
- Control statements help to jump control from one part of the program to another depending on whether a particular condition is satisfied or not.
- C programming language provide three types of control statements.
  - –Sequence Control Statements.
  - –Selection or Decision Control Statements.
  - –Repetition or Loop Control Statements.



1. Introduction to Decision Control Statements

- Normally instructions in the program are executed in the same order in which they appear in the program. But in some cases we need to execute a set of instructions based on certain criteria.
- If you look at our daily life we all need to alter our actions according to the changing situation.
- For example if the weather is sunny, then you will prefer to have cold drinks or juice but if the weather is cold, then you will prefer to have coffee or hot soup.
- As you notice that the above decision depends on the condition of weather.
- In the same way as above, sometimes depending on the situation we may want to execute one set of instructions in one situation and in some other situation we may want to execute another set of instructions.
- This way of controlled execution of statements can be achieved by using decision statements in the program, hence they are called as control statements.

1. Introduction to Decision Control Statements…contd.,

- Execution of different sets of instructions at different times is totally based on relational/logical condition.
- Relational/Logical expression returns either true or false. If the test expression (condition) returns non-null and non-zero, then it is assumed as true, and if it is either zero or null, then it is assumed as false value.
- Decision making statements use the outcome of Relational/Logical expression (true/false) and based on outcome, a specific set of instructions is executed.
- This allows a program to take different courses of actions depending on different conditions.
- C language has such decision making capabilities and supports the following statements known as control or decision making statements.
  i. **if**
  ii. **if-else**
  iii. **if-else-if**
  iv. **switch-case**

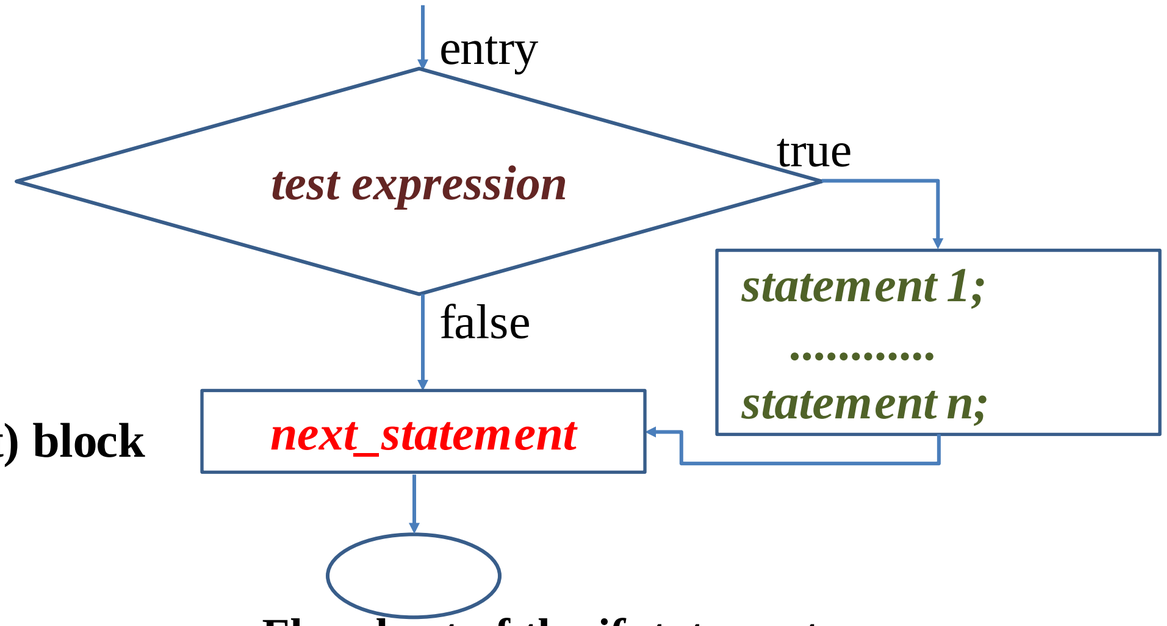
2. i) *if* statement

- When you want to execute a statement or statement block based on condition, then **if** statement can be used.

Syntax of the *if*:

```
if (test expression)
{
        statement 1;
        ............
        statement n;
}
next_statement;
```

If (statement) block



Flowchart of the *if* statement.

- Use of curly brackets ({ }) for the ***statement block*** is optional in case if ***statement block*** is having only a single statement.

if statement….contd.,


```
Syntax of the if:
 if (test expression)
 {
        statement 1;
        ............
        statement n;
 }
next_statement;
```

- In the syntax, ***if*** is the keyword, and ***test expression(condition)*** is an expression that returns boolean value (true/false i.e., 1/0 (example., a<b, a!=0, a>b && b>c etc.,)).
- If ***test expression*** is true, then ***statement/statement block*** immediately followed by ***if*** is executed, otherwise ***statement/statement block*** is ignored.
- Then control passes from the ***if*** statement to the ***next_statement*** in the program unless one of the statements contains a **break**, **continue**, or **goto** statement.
- Note that there is no semicolon after the ***test expression***. This is because the condition and statement should be put together as a single statement.

Example for the *if* statement


```
#include<stdio.h>
int main()
{
int x=10;
if (x>0) x++;
printf("\n x = %d", x);
return 0;
}
```

Same as

```
#include<stdio.h>
int main()
{
int x=10;
if (x>0)
{
x++;
}
printf("\n x = %d", x);
return 0;
}
```

test expression

statement block

next_statement

Example - Program to increment and print a 'x', if value of x is more than 10.

- Program given on the left side, we take a variable '**x**' and initialize it to 10.
- In the test expression, we check if the value of '**x**' is greater than 0.
- As **10 > 0**, the ***test expression (x>0)*** evaluates to true, and the value of '**x**' is incremented (x++, i.e., x=11).
- Then the value of '**x**' is printed on the screen. The output of this program is 11.
- Observe that the printf("\n x = %d", x); statement will be executed even if the ***test expression*** is false.

Additional examples for the *if* statement

Example 1. Write a Program to check if withdraw amount is more than balance or not, if withdraw amount is more than balance, then notify user for insufficient balance.

```
#include<stdio.h>
main()
{
 int balance=20000,w_amount;
 printf("enter amount to withdraw: "); /*ask user to
enter withdraw amount*/
 scanf("%d",&w_amount); //read withdraw amount
 if(w_amount>balance) /*check if withdraw amount is
more than balance*/
 {
/* if withdraw amount is more than balance, notify him
insufficient balance*/
  printf("Sorry, insufficient balance.!!!");
 }
//otherwise do nothing and exit the program normally
 return 0;
}
```

Example 2. Write a Program to ccompute income tax based on income and print net income tax to be paid based on income.

```
#include<stdio.h>
int main(){
    float tax = 0, income;
    printf("Enter your income\n");
    scanf("%f", &income);
    if(income>=250000 && income<=500000){
        tax = tax + 0.05 * (income - 250000);
    }
    if (income >= 500000 && income <= 1000000)
    {
        tax = tax + 0.20 * (income - 500000);
    }
    if (income >= 1000000)
    {
        tax = tax + 0.30 * (income - 1000000);
    }
    printf("Your net income tax to be paid is: %f\n", tax);
    return 0;
}
```

Example 3. Write a Program to check he/she is passed or failed based on input marks.

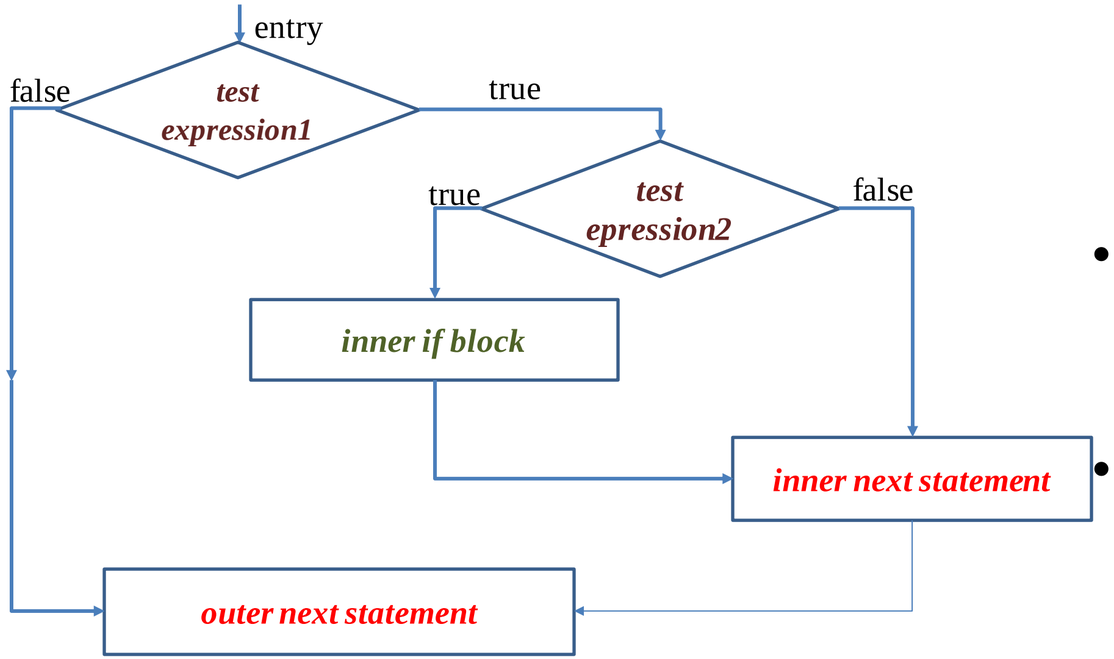
```
#include<stdio.h>
main()
{
 int marks;
 printf("enter your marks"); //ask user to enter marks
 scanf("%d",&marks);//read marks into memory
 if(marks>50) //check if value of marks is greater than 50
 {
  //if marks is greater than 50, print “you are passed”
  printf("you are passed");
 }
 if(marks<50) //check if value of marks is less than 50
 {
   //if marks is less than 50, print “you are failed”
  printf("you are failed");
 }
   //otherwise do nothing and exit program normally
 return 0;
}
```

Example 4. Write a Program to check whether he/she is eligible to retire or not.

```
#include<stdio.h>
main()
{
 int age;;
 printf(" enter your age: " ); //ask user to enter his/her age
 scanf("%d",&age); //read age
 if(age>=60) //check if his/her age is more than 59
 {
/* if age is more than 59, then notify user “you can retire
now*/
  printf("you can retire now");
 }
//otherwise do nothing and exit the program normally
 return 0;
}
```

Nested *if* statement


```
Syntax of the nested if:
if(test expression1)
{
        if(test expression2)
        {
         inner if block;
        }
        inner next statement;
}
outer next statement;
```



Flowchart of the nested *if* statement.

- Use of ***if*** within the body of another ***if*** is called as nested ***if*.**
- When a series of conditions are to be evaluated for the value of a variable, we may have to nest an ***if*** statement, i.e., need to use ***inner if*** inside the ***outer if***.
- As per syntax of nested ***if***, if the ***test expression1*** is true it continues to perform the second test (***test expression2***). Otherwise it ignores the ***inner if*** and continues to execute the ***outer next statement***.
- If the ***test expression2*** is true, it continues to execute the ***inner if block***, otherwise control jumps to the ***inner next statement*** and it is executed.
- After executing the ***inner next statement*** then the control is transferred to the ***outer next statement*** then it is executed and reaches the end of the program..

Example for the Nested *if* statement

- In the following example of nested ***if***, the ***test expression1 (no<0)*** is evaluated first, if the condition returns true, it continues with execution of ***inner if.*** Otherwise control is transferred to printf("\n Outer if is ignored."); statement and it is executed.
- During the execution of ***inner if,*** the ***test expression2 (no%2==0)*** is evaluated and if this condition is true, then the control is transferred to **printf("\n The given number is positive odd number.");** statement and it is executed.

```
#include<stdio.h>
int main() {
int no;
printf("\n Enter a number: ");
scanf("%d", &no);
if(no>0)
{
    if(no%2==0)
    {
        printf("\n The given number is positive odd number.");
    }
    printf("\n Inner if is ignored.");
}
printf("\n Outer if is ignored.");
return 0;
}
```

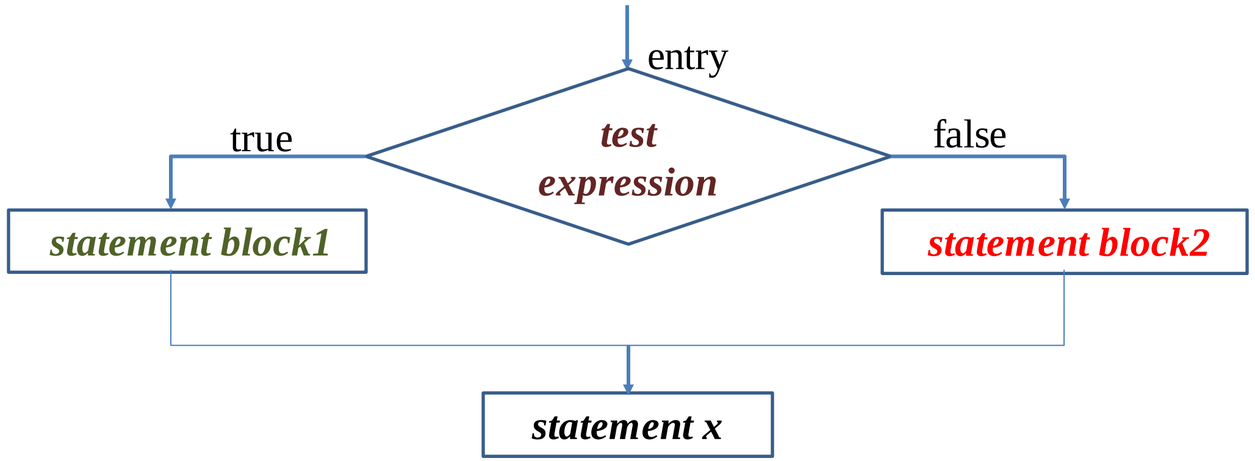
Example - Program to check input number is positive odd number or not..

- Otherwise, the condition of ***inner if*** is false, then control is transferred to printf("\n Inner if is ignored."); statement and it is executed.
- Now the control is shifted to printf("\n Outer if is ignored."); and then it is executed and it reaches the end of the program.

2. ii) *if-else* statement

- In case of ***if*** statement, when the ***test expression*** is evaluated to true, the ***statement block1*** followed by the ***if*** statement is executed, otherwise the ***statement block1*** is skipped by the compiler.
- ***if-else*** is an extension of the 'simple ***if***' statement and suppose if u want to execute a separate block of statements in case ***if*** condition returns false then we have to use ***if-else***.

```
Syntax of the if-else:
   if(test expression)
   {
      statement block1;
   }
   else
   {
      statement block2;
   }
   statement x;
```



Flowchart of the *if-else* statement.

- In the above syntax, if the ***test expression*** is true then the ***statement block1***, immediately following the if statement is executed, otherwise the ***statement block2*** is executed. Then control is shifted to ***statement x.***

Example for the *if-else* statement

```
Syntax of the if-else:
   if(test expression)
   {
      statement block1;
   }
   else
   {
      statement block2;
   }
   statement x;
```

- In either case, either ***statement block1*** or ***statement block2*** will be executed, but not both.
- Control is then passed from the ***if*** statement to the ***statement x*** in the program unless one of the statements contains a **break**, **continue**, or **goto**.
- After executing either ***statement block1*** or *2*, the control will move to ***statement x***. Therefore, ***statement x*** is executed in every case.

Example,

```
#include<stdio.h>
int main()
{
    int a;
    printf("\n Enter the value of a : ");
    scanf("%d", &a);
    if(a%2==0)
            printf("\n %d is even", a);
    else
            printf("\n %d is odd", a);
    return 0;
}
```

- In the program given on the left side, we declare variable '**a**' of int type and read the value of '**a**' from the user.
- Now the control moves to the if statement, which checks whether the condition (**a%2==0**) is true or false.
- if condition (**a%2==0**) is true**,** then **printf("\n %d is even", a);** is executed.
- Otherwise, **printf("\n %d is odd", a);** is executed.
- Now control moves to statement **return 0;** after if-else blocks;

Example - Program to check input number is odd or even..

Additional examples for the *if-else* statement

Example 1: Write a Program to check age of the user, if age is above 17 print "You can vote" other print "You cant vote".

```
#include<stdio.h>
main()
{
 int age;
 printf("enter your age"); //ask user to enter his/her age
 scanf("%d",&age); //read age
 if(age>17) //check if age is greater than 17
 {
/*if age is greater than 17, then display "you are eligible to vote"*/
  printf("you are eligible to vote");
 }
else
//otherwise, display "you are not eligible to vote"
 printf("you are not eligible to vote");
 return 0;
}
```

Example 2: Write a Program to check age of the user, if user age is above 17 and below 71, then print "You can drive", otherwise print "You cant drive".

```
#include<stdio.h>
int main()
{
 int age;
 printf("Enter your age\n");//ask user to enter his/her age
 scanf("%d", &age); //read age
 if(age <= 70 && age>=18) /*check if age is above 18 and bellow 70*/
  {
/*if age is above 18 and bellow 70, print You are above 18 and below 70, you can drive*/
   printf("You are above 18 and below 70, you can drive\n");
  }
 else
  {
//otherwise print You can't drive
   printf("You can't drive\n");
  }
 return 0;
}
```

Example 3: Write a program to check input character is uppercase character or lowercase character.

```
#include<stdio.h>
int main()
{
 // 97-122 = a-z ASCII Values
 char ch;
 printf("Enter the character\n");
 scanf("%c", &ch);
 if(ch<=122 && ch>=97)
  {
   printf("It is lowercase character");
  }
 else
  {
   printf("It is not lowercase character");
  }
 return 0;
}
```

Example 4: Write a program to check percentage of marks scored, if percentage marks is less than 40 or Physics, chemistry and maths marks are less than 33, then print failed, otherwise print passed.

```
#include<stdio.h>
int main(){
   int physics, chemistry, maths;
   float total;
   printf("Enter Physics Marks\n");
   scanf("%d", &physics);

   printf("Enter Chemistry Marks\n");
   scanf("%d", &chemistry);

   printf("Enter Maths Marks\n");
   scanf("%d", &maths);
   total = (physics + maths + chemistry)/3;
   if((total<40) || physics<33 || maths<33 || chemistry<33){
      printf("Your total percentage is %f and you are fail\n", total);
   }
   else{
      printf("Your total percentage is %f and you are pass\n", total);
   }
   return 0;
}
```

Nested *if-else* statement

Syntax of the nested *if-else*:

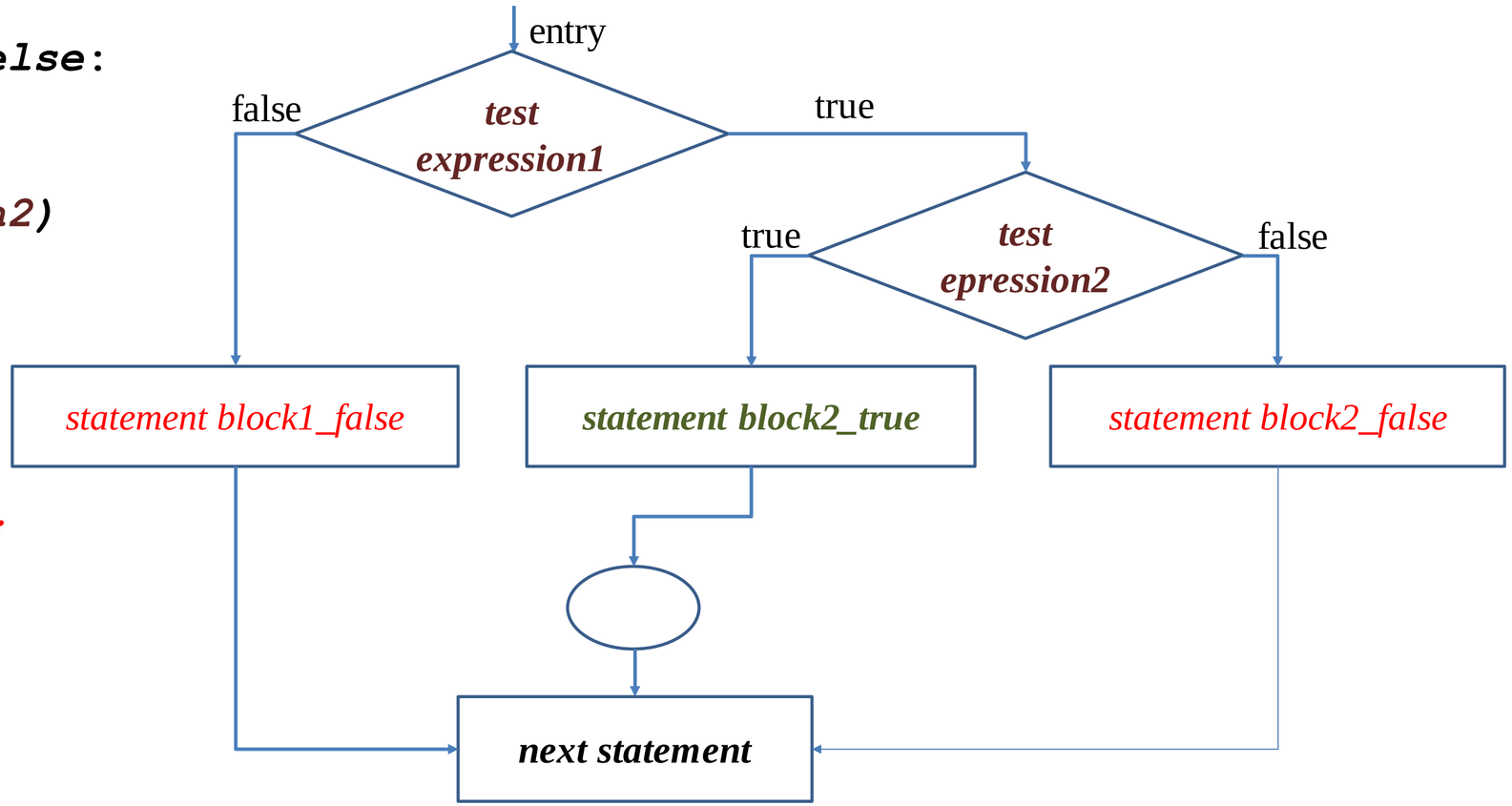
```
if(test expression1)
{
        if(test expression2)
        {
     statement block2_true;
        }
        else
        {
     statement block2_false;
        }
}
else
{
   statement block1_false;
}
next_statement;
```

- Use of ***if-else*** within the body of another ***if-else*** is called as the nested ***if-else*.**
- When a series of decisions are involved, we may have to use more than one ***if-else*** statement.
- As per the syntax of ***if-else***, if the ***test expression1*** is false, then the ***statement block1_false*** will *be* executed otherwise it continues to perform the second test (***test expression2***).
- If the ***test expression2*** returns true, the ***statement block2_true*** will be executed, otherwise ***statement block2_false*** is executed.
- Then the control is transferred from the ***inner if-else*** to the ***next_statement*** after the ***outer if-else*.**

Nested *if-else* statement….contd.,

Syntax of the nested *if-else*:

```
if(test expression1)
{
        if(test expression2)
        {
   statement block2_true;
        }
        else
        {
   statement block2_false;
        }
}
else
{
statement block1_false;
}
next statement;
```



Flowchart of the nested *if-else* statement.

Additional examples for the nested *if-else* statement

Example 1: Write a program to check the input number is positive or not, if positive then check input is odd or even number.

```
#include<stdio.h>
void main()
{
int number;
printf("Enter a Number\n");
scanf("%d",&number);
if(number>0)
{
    if(number%2==0) {
    printf("%d is even. ",number);
    }
    else
    {
    printf("%d is odd. ",number);
    }
}
else {
printf("enter a valid positive number") ;
}
return 0;
}
```

Example 2: Write a program to find biggest among three numbers.

```
#include <stdio.h>
int main ()
{
    int x, y, z;
    printf ("Enter three number \n");
    scanf ("%d %d %d", &x, &y, &z);
    if (x>y) // first condition
    {
//The code of this block will run when the first condition is true
        if (x>z) // second condition
            printf ("Greater value is: %d", x);
        else
            printf ("Greater value is: %d", z);
    }
   else    {
 /*If the first condition is False, then the code of this block will run.*/
     if (y>z)
         printf ("Greater value is: %d", y);
      else
         printf ("Greater value is: %d", z);
   }
   return 0;
}
```

Example 3: Write a program to check based on age of user, whether user is minor or senior citizen or eligible worker..

```
#include <stdio.h>
int main() {
 int age;
 printf("Please Enter Your Age Here:\n");
 scanf("%d",&age);
 if ( age < 18 ) {
  printf("You are Minor.\n");
  printf("Not Eligible to Work");
 }
 else {
  if (age >= 18 && age <= 60 ) {
    printf("You are Eligible to Work \n");
    printf("Please fill in your details and apply\n");
   }
  else {
    printf("You are too old to work as per the Government
rules\n");
    printf("Please Collect your pension! \n");
   }
 }
 return 0;
}
```

Example 3: Write a program to check given three numbers are equal or not.

```
#include <stdio.h>
int main() {
  // variables to store the three numbers
  int a, b, c;
  //take input from the user
  scanf("%d %d %d", &a, &b, &c);
  //if else condition to check whether first two numbers are equal
  if (a == b) {
    //nested if else condition to check if c is equal to a and b
    if (a == c) {
      //all are equal
      printf("Yes");
    } else {
      //all are not equal
      printf("No");
    }
  } else {
    //the first two numbers are not equal, so they are not equal
    printf("No");
  }
  return 0;
}
```

2. iii) *if-else-if* statement

Syntax of the *if-else-if*:

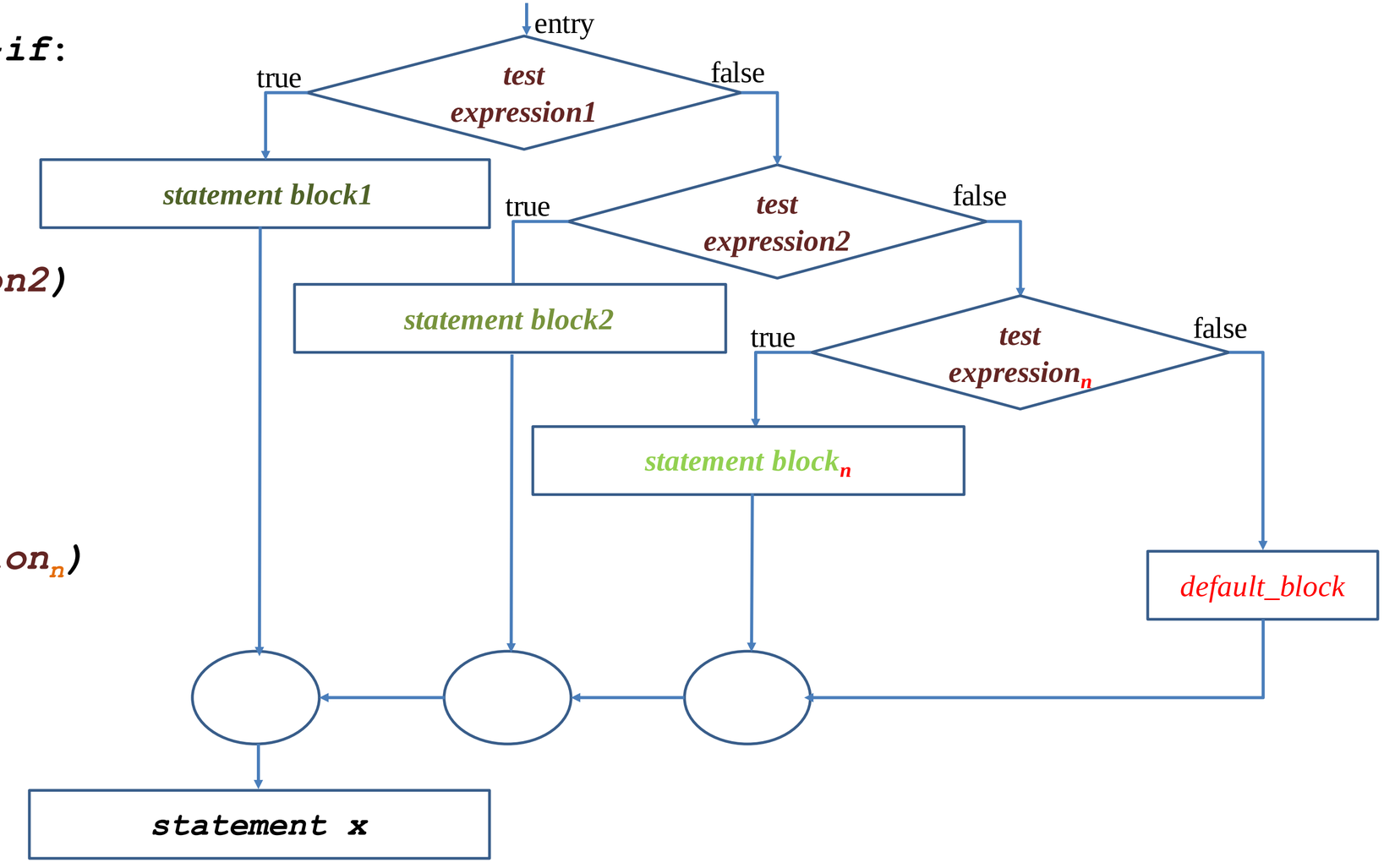
```
if(test expression1)
  {
statement block1;
  }
else if(test expression2)
  {
 statement block2;
  }
 .
 .
 else if(test expression_n)
  {
 statement block_n;
  }
else
 {
   default_block;
 }
statement x;
```

- ***if–else-if*** is also called as ***else-if*** ladder and used to test additional conditions apart from the initial test expression.
- When there are more than two test expressions (***test expression1, test expression2…. test expression$_n$***) to be evaluated, we need to use the ***else-if*** ladder.
- As it is given in the syntax ***else-if*** ladder is a chain of ***if***'s in which the statement associated with each ***else*** is an ***if*** (i.e., ***else-if***). And ***if, else-if, else*** all these are having statement blocks associated with them.
- As soon as a true condition is found, the statement block (***statement block1 or statement block2 or statement block$_n$***) associated with it is executed and the control is transferred to ***statement x***.
- When all the conditions become false, then the final ***else*** containing the ***default_block*** will be executed. Then control is transferred to ***statement x.***

if-else-if statement….contd.,

Syntax of the *if-else-if*:

```
if(test expression1)
  {
statement block1;
  }
else if(test expression2)
  {
 statement block2;
  }
 .
 .
 else if(test expression_n)
  {
 statement block_n;
  }
else
 {
   default_block;
 }
statement x;
```



Flowchart of the *if-else-if* statement.

Example for the *if-else-if* statement

- C supports simple ***if*** statements., so it is not necessary that every ***if*** statement should have an ***else*** block.
- After the first test expression, the programmer can have as many ***else–if*** branches as he wants depending on the expressions that have to be tested.

```
#include
int main()
{
  int num;
  printf("\n Enter any number : ");
  scanf("%d", &num);
  if(num==0)
    printf("\n The value is equal to zero");
  else if(num>0)
    printf("\n The number is positive");
  else
    printf("\n The number is negative");
  return 0;
}
```

Example - Program to check the given number is positive or negative or equal to zero.

- In the following example, the program tests that the given number is equal to zero, positive, or negative.
- Note that if the first test expression (num==0) evaluates to a true value, then the rest of the statements in the code will be ignored after executing **printf("\n The value is equal to zero");** statement.
- Otherwise if the first test expression (num==0) returns false, test expression (num>0) is evaluated. If test expression num>0 is true, **printf("\n The number is positive");** statement is executed. Otherwise it executes printf("\n The number is negative"); statement.
- Then the control will jump to **return 0** statement.

Example 1: Write a program to print grade of the user based on the input marks.

```
#include<stdio.h>
void main()
{
    int marks;
    printf("Enter your marks ");
    scanf("%d",&marks);
    if(marks<0 || marks>100)
    {
        printf("Wrong Entry");
    }
    else if(marks<50)
    {
        printf("Grade F");
    }
    else if(marks>=50 && marks<60)
    {
        printf("Grade D");
    }
```

```
else if(marks>=60 && marks<70)
    {
        printf("Grade C");
    }
    else if(marks>=70 && marks<80)
    {
        printf("Grade B");
    }
    else if(marks>=80 && marks<90)
    {
        printf("Grade A");
    }
    else
    {
        printf("Grade A+");
    }
}
```

Example 2: Write a program to compute Electricity bill based on total units consumed.

```
#include<stdio.h>
void main()
{
   int units;
   float e_bill;
   printf("Enter number of units consumed: ");
   scanf("%d",&units);
   if(units>0 && units<=100)
   {
      e_bill=units*2+15;
   }
   else if(units >100 && units<=200)
   {
      e_bill=units*3+25;
   }
   else if(units>200 && units<=300)
   {
       e_bill=units*3+35;
   }
   else
   {
      e_bill=units*5+50;;
   }
 printf("\nThe electricity billing amount is Rs. %f ", e_bill);
}
```

Example 3: Write a program to compare two numbers n1 and n2 and find n1 is equal to n2 or n1 is less than n2 or n1 more than n2.

```
#include<stdio.h>
void main()
{
   int n1,n2;
   printf("Enter two numbers: ");
   scanf("%d %d",&n1,&n2);
   if(n1 == n2)
   {
      printf("Both %d and %d are equal. ", n1, n2 );
   }
   else if(n1>n2)
   {
      printf( " %d is greater than %d ", n1,n2 );
   }
   else if(n1<n2)
   {
     printf(" %d is smaller than %d ", n1, n2 );
   }
   else
   {
     printf(",%d and %d are not related", n1, n2 );
   }
}
```

Example for the nested *if-else-if* statement

- ***if-else-if*** inside another ***if-else-if*** construct is called as nested ***if-else-if***.

```
#include<stdio.h>
void main()
{
   int college,program;
   printf("\n1. College A\n2. College B\n3. Exit\n");
   printf("\nEnter your college: ");
   scanf("%d",&college);
   if(college==1)
   {
      printf("\n1. Program X1\n2. Program Y1\n3. Program Z1");
      printf("\nEnter your choice: ");
      scanf("%d",&program);
      if(program==1)
         printf("\nYour CET rank should be [1-100] to get into Program X1.");
      else if(program==2)
         printf("\nYour CET rank should be [101-500]to get into Program X1.");
      else if(program==3)
         printf("\nYour CET rank should be [501-999]to get into Program X1.");
      else
         printf("\nEnter valid program [1-3]");
   }
   else if(college==2)
      printf("There are no seats available, good luck for next year!!!");
   else
      exit(0);
}
```

```
else if(college==2)
   {
      printf("\n1. Program X2\n2. Program Y2\n3. Program Z2");
      printf("\nEnter your choice: ");
      scanf("%d",&program);
      if(program==1)
      {
         printf("\nYour CET rank should be [1000-2000] to get into program X2.");
      }
      else if(program==2)
      {
         printf("\nYour CET rank should be [2001-5000]to get into program X2.");
      }
      else if(program==3)
      {
         printf("\nYour CET rank should be [5001-9999]to get into program X2.");
      }
      else
      {
         printf("\nEnter valid program [1-3]");
      }
   }
   else
   {
      printf("\nEnter valid college [1-5]");
   }
}
```

2. iv) ***switch-case*** statement

- A ***switch-case*** statement is a multi-way decision statement that is a simplified version of an ***if– else–if*** block.
- Switch is a control statement that allows a value of any variable to change control of execution and switch makes one selection when there are several choices to be made. As an alternative to ***switch-case*** still we can use an ***if-else-if*** statement.
- But use of ***if-else-if*** is not suggested if the number of alternative conditions increases as it leads to increase in complexity of the program.
- In ***switch-case*** statement the values of a given variable (or expression) are tested against a list of ***case constants*** or ***constant expressions***(case values) and when a match is found a block of statements associated with that case is executed.
- ***switch-case*** is mostly used when there is only one variable to evaluate in the expression and as ***switch-case*** statement compare the value of variable to only integer case constant, so the value of variable must be an integer or character type.
- The ***default*** label is optional and it is executed only when the value of expression does not match with any of the ***case*** constants or ***expression*** constants. The ***default*** label can be placed anywhere in the ***switch-case*** construct.

switch-case statement….contd.,

- Compared to ***switch-case***, one advantage of using ***if–else–if*** statement is that it can evaluate more than one expression in a single logical structure and value of variable or expression not necessarily be an integer type or character type.
- Advantages of Using a ***switch-case*** statement ***Switch–case*** statement is preferred by programmers due to the following reasons:
  - Easy to debug
  - Easy to read and understand
  - Ease of maintenance as compared to its equivalent ***if–else*** statements
  - Like ***if–else*** statements, switch statements can also be nested
  - Executes faster than its equivalent ***if–else*** construct

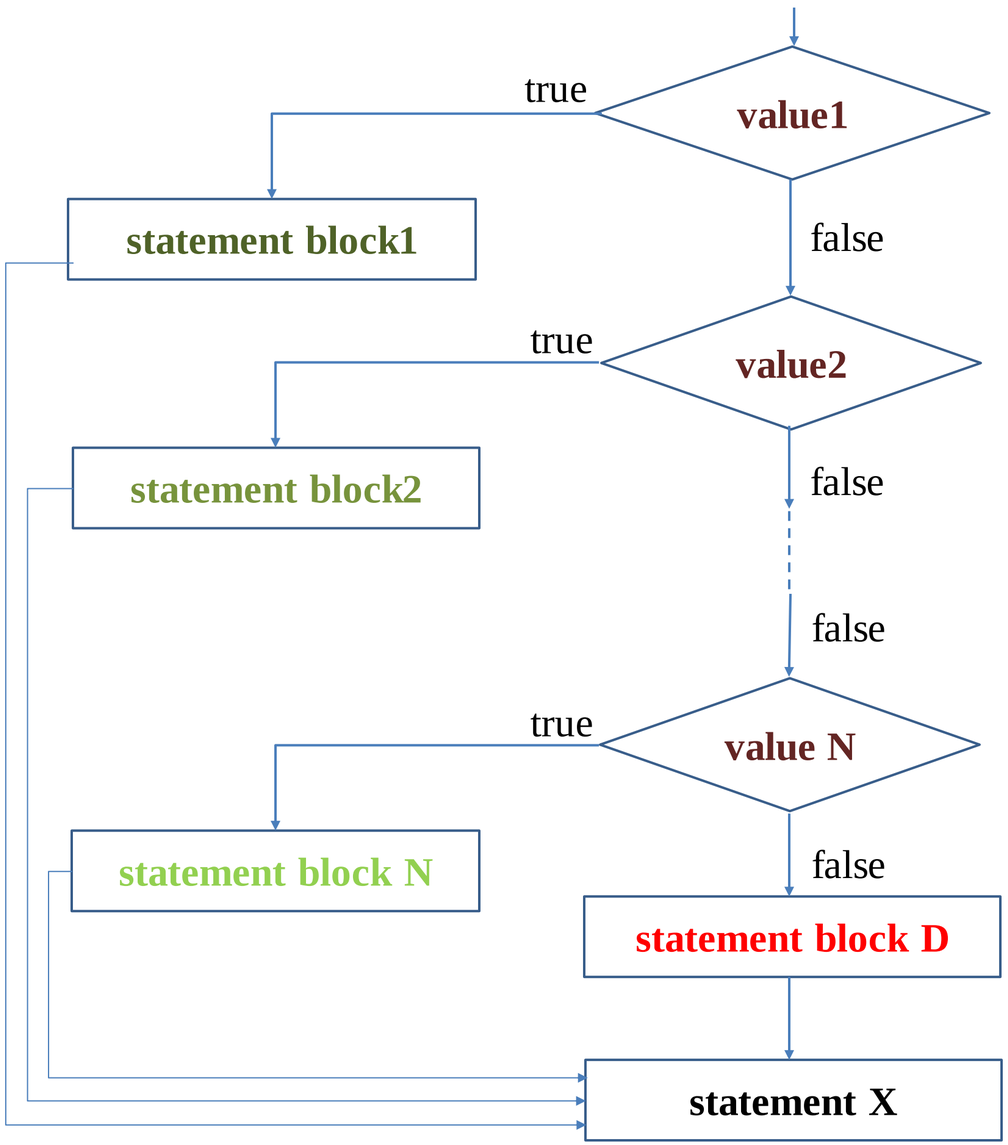
switch-case statement….contd.,


```
Syntax of the
switch-case:
switch (variable)
{
case value1:
statement block1;
break;
case value2:
statement block2;
break;
.....................
case value N:
statement block N;
break;
default:
statement block D;
}
statement X;
```

- As it is given in the syntax, the input value of ***variable*** is compared with case values (***value1, value2….value N***) based on the match with specific case value, statement block (***statement block1 or statement block2 or statement block N***) followed by that ***case*** is going to be executed.
- ***default*** is an optional ***case*** that is executed when the value of the ***variable*** does not match with any of the ***case*** values and it is always recommended to include it, as it handles any unexpected ***case***.
- In the syntax note that, there is another keyword called ***break***, it must be used at the end of each ***case,*** this is because if it is not used, then the ***case*** that is matched along with all the following ***cases*** till next ***break*** will be executed.
- The ***break*** statement tells the compiler to jump out of the ***switch-case*** construct after executing the corresponding statement block of a matched ***case*** and pass the control to ***statement X***.

switch-case statement….contd.,


```
Syntax of the switch-case:
switch (variable)
{
case value1:
statement block1;
break;
case value2:
statement block2;
break;
.....................
case value N:
statement block N;
break;
default:
statement block D;
}
statement X;
```



Flowchart of the nested *switch-case* statement.

Example for the ***switch-case*** statement

```
#include <stdio.h>
int main()
{
char ch;
printf("\n Enter any character : ");
scanf("%c", &ch);
switch(ch)
{
 case 'A':
 case 'a':
 printf("\n %c is VOWEL", ch);
 break;
 case 'E':
 case 'e':
 printf("\n %c is VOWEL", ch);
 break;
 case 'I':
 case 'i':
 printf("\n %c is VOWEL", ch);
 break;
 case 'O':
 case 'o':
 printf("\n %c is VOWEL", ch);
 break;
 case 'U':
 case 'u':
 printf("\n %c is VOWEL", ch);
 break;
 default: printf("\n %c is not a
vowel", ch);
}
return 0;
}
```

Example C Program to check the given character is vowel or not.

- In the example, we declare the variable '**ch**' of char type, after reading the '**ch**' value, it is compared with ***case*** constants (case values i.e., A, a, E, e, I, i, O, o, U, u) of ***case*** statements.
- Once the value of '**ch**' matches to any of the ***case*** constants, a block of statements immediately followed by the corresponding ***case*** statement is executed.
- In the example, there is no ***break*** after the ***case*** 'A', if the value of '**ch**' matches 'A', then all the ***case*** blocks till the next ***break*** statement are executed and then control exit from the ***switch-case*** construct.
- If the value of '**ch**' does not matches with any of the ***case*** constants, then the program prints that the character is not a vowel (***default*** case).

Additional examples for the ***switch-case*** statement

Example 1: Write a program to perform arithmetic operation based on the user choice.

```
#include<stdio.h>
void main()
{
   int n1,n2,choice;
   float result;
   printf("\nEnter numbers n1 and n2: ");
   scanf("%d %d",&n1, &n2);
   printf("\n1. Addition\n2. Subtraction\n3. Multiplication\n4. Division\n5.
Exit\n");
   printf("\nEnter your choice: ");
   scanf("%d",&choice);
   switch(choice)
   {
   case 1:
      result=n1+n2;
      break;
   case 2:
      result=n1-n2;
      break;
```

```
 case 3:
      result=n1*n2;
      break;
   case 4:
      result=(float)n1/(float)n2;
      break;
   case 5:
      exit(0);
   default:
      printf("\nenter valid choice [1 - 5]");
   }
   printf("\nresult is %lf",result);
   return 0;
}
```

Example 2: Write a program to display marks range to be secured to obtain input grade.

```
#include<stdio.h>
void main()
{
    char grade;
    printf("\nEnter your Grade: ");
    scanf("%c",&grade);
    switch(grade)
    {
    case 'S':
    case 's':
        printf("Marks range to secure S grade is: 90-100");
        break;
    case 'A':
    case 'a':
        printf("Marks range to secure A grade is: 75-89");
        break;
    case 'B':
    case 'b':
        printf("Marks range to secure B grade is: 65-74");
        break;
```

```
case 'C':
    case 'c':
        printf("Marks range to secure C grade is: 55-64");
        break;
    case 'D':
    case 'd':
        printf("Marks range to secure D grade is: 50-54");
        break;
    case 'E':
    case 'e':
        printf("Marks range to secure E grade is: 40-49");
        break;
    case 'F':
    case 'f':
        printf("Marks range to secure F grade is: 00-39");
        break;
    default:
        printf("\nEnter valid Grade [A|a - F|f]: ");
    }
    return 0;
}
```

Example 3: Write a program to display Rank range based on college choice. [note that you have to assume hypothetical college names and rank range.]

```
#include<stdio.h>
void main()
{
   int choice;
   printf("\n1. College A\n2. College B\n3. College C\n4. College D\n5. College E");
   printf("\nEnter your choice: ");
   scanf("%d",&choice);
   switch(choice)
   {
   case 1:
      printf("\nYour CET rank should be <1000.");
      break;
   case 2:
      printf("\nYour CET rank should be [1000 - 9999].");
      break;
   case 3:
      printf("\nYour CET rank should be [10000-19999].");
      break;
```

```
 case 4:
      printf("\nYour CET rank should be [20000-39999].");
      break;
   case 5:
printf("\nYour CET rank should be [30000-49999].");
      break;
   default:
     printf("\nYour wont get seat in any college".);
   }
   return 0;
}
```

Example for the nested ***switch-case*** statement

The ***inner switch-case*** inside an ***outer switch-case*** is called a nested ***switch-case*** and we can nest any number of ***switch-case*** statements, there is no restriction for it. The ***case*** constants of the ***inner*** and ***outer switch*** may have common ***values***.

```
#include <stdio.h>
int main() {
    int ID = 500;
    int password = 000;
    printf("Plese Enter Your ID:\n ");
    scanf("%d", & ID);
    switch (ID) {
      case 500: //ID==500
        printf("Enter your password:\n ");
        scanf("%d", & password);
        switch (password) {
          case 000: //password==000
            printf("Welcome dear Programmer\n");
            break;
          default:
            printf("incorrect password");
            break;
        }
        break;
      default:
        printf("incorrect ID");
        break;
    }
}
```

Example C Program to evaluate hypothetical username and password.

- In the example, the program asks the user to type his ***ID***, and if the ***ID*** is valid then it will ask him to enter his ***password***.
- If the ***password*** is correct the program will print "Welcome dear Programmer", otherwise, the program will print Incorrect ***Password*** and if the ***ID*** does not exist, the program will print Incorrect ***ID***.
- An ***outer switch*** construct is used to compare the value entered in variable ***ID***. It execute the ***statement block*** associated with the matched case(when ***ID==500***).
- If the ***statement block*** is executed with the matched ***case***, an ***inner switch*** is used to compare the values entered in the variable ***password*** and execute the statements linked with the matched ***case***(when ***password==000***).
- Otherwise it executes the ***default*** case of the ***inner switch-case*** construct.

Example of the nested ***switch-case***…contd.,

Example 1: Write a program to display Rank range program wise based on college choice. [note that you have to assume hypothetical college names and rank range.]

```
#include<stdio.h>
void main()
{
   int college,program;
   printf("\n1. College A\n2. College B\n");
   printf("\nEnter your choice: ");
   scanf("%d",&college);
   switch(college)
   {
   case 1:
      printf("\n1. Program X1\n2. Program Y1\n3. Program Z1");
      printf("\nEnter your choice: ");
      scanf("%d",&program);
      switch(program)
      {
      case 1:
         printf("\nYour CET rank should be [1-100].");
         break;
      case 2:
         printf("\nYour CET rank should be [101-500].");
         break;
      case 3:
         printf("\nYour CET rank should be [501-999].");
         break;
      default:
         printf("\nEnter valid program [1-3]");
      }
```

```
break;
case 2:
      printf("\n1. Program X2\n2. Program Y2\n3. Program
Z2");
      printf("\nEnter your choice: ");
      scanf("%d",&program);
      switch(program)
      {
      case 1:
         printf("\nYour CET rank should be [1000-2000].");
         break;
      case 2:
         printf("\nYour CET rank should be [2001-5000].");
         break;
      case 3:
         printf("\nYour CET rank should be [5001-9999].");
         break;
      default:
         printf("\nEnter valid program [1-3]");
      }
      break;
   default:
      printf("\nEnter valid college [1-5]");
   }
   return 0;
}
```

3. Iterative Statements

- Apart from conditional statements, programming languages usually supports another important control structures called iterative statements (loop/repetitive statements) and they are used to repeat the execution of a sequence of statements (statement block) as long specified condition remains true.
- A loop is one of the most fundamental logical structures in computer programming that allow computers to repeat specific task such as,
  - *Computing Fibonacci series up to n (n is a certain number), Printing of "R V College of Engineering" 10 times, Computing n prime numbers, Playing of favorite song n times, Computing area of 100 circles with different radii* and many more.,
- Loop structure requires *loop control variable* and three important expressions viz., *initialization*, *test expression*, *increment/decrement* and every loop statement is followed by a sequence of statements called as the statement block.
- The execution of the statement block is repeated as long as the test expression (condition) returns true and stops only after the test expression returns false.
- Value of a loop variable acts as a controlling factor for the repetition of execution of the statement block, hence the loop variable is usually called as the control variable for the loop.

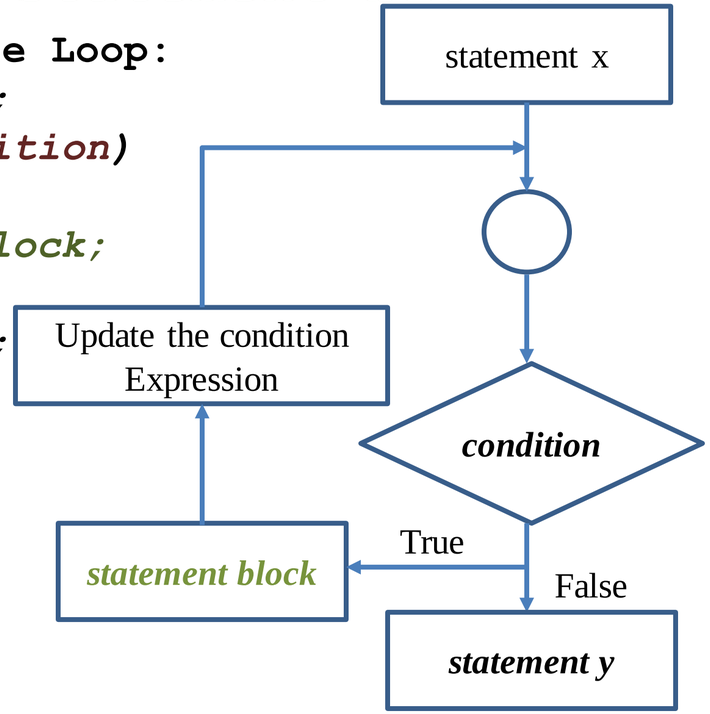
Iterative Statements….contd.,

- Loops are classified as entry controlled Loops and exit controlled Loops, entry controlled Loops are those in which test expression is evaluated before execution of loop block (statement block). Example: ***while*** and ***for***.
- Whereas exit controlled Loops are those in which test expression is evaluated only after execution of loop block. Example ***do-while.***
- Key difference between entry controlled Loop and exit controlled Loop is that
  - Entry loop, loop block won't execute if test expression returns false., whereas in case of exit loop, loop block will be executed even though test expression returns false.
- Loops can also be classified as definite loops and indefinite loops, definite loop is one in which number of repetitive executions (iterations) of loop block is known. Example ***for***.
- On other hand indefinite loop is the one in which number of iterations are not known, they keep repeating execution of loop block as long as test expression of loop returns true. Example: ***while*** and ***do-while***.
- C supports all the three types of loop statements i.e., it supports ***while***, ***do-while*** and ***for*** statements.

3. i) *while* Loop

- The while loop provides a mechanism to repeat the ***statement block*** as long as test the ***condition*** returns true.
- As it is given in the syntax, note that in the while loop, the ***condition*** is tested before the execution of any of the statements in the ***statement block.***
- If the ***condition*** is true, only then the ***statement block*** will be executed, otherwise if the condition is false, the control will jump to ***statement y***, that is the immediate statement outside the ***statement block***.

```
Syntax of while Loop:
   statement x;
   while (condition)
   {
   statement block;
   }
   statement y;
```



Flowchart of the *while* loop.

- The value of the while loop variable that is tested with ***condition*** determines when the loop will end.
- In order to continue execution of the ***statement block*** we need to constantly update the *loop control variable* of the while loop either by incrementing or decrementing its value.
- Note that if the loop variable is not updated, then the ***condition*** never becomes false, as a result loop becomes infinite loop.

Example of *while* Loop

- For example, the following code prints the first 10 numbers using a while loop. In the program '**i**' is the loop variable that is initialized to 1.
- Note that initially '**i**' is less than 10, i.e., the condition (**i<=10)** in the loop returns true, so control is moved to ***statement block*** and executes it i.e., prints value of '**i**' (**printf("\n %d", i);)** and update the '**i**' value (**i = i + 1;**).

```
#include <stdio.h>
int main()
{
   int i = 1;
   while(i<=10)
   {
      printf("\n %d", i);
      i = i + 1; // condition updated
   }
   return 0;
}
```

Example C Program to print first 10 serial numbers.

- Since we are still in the while loop, after increasing the '**i**' value by 1, again the condition is evaluated with the updated '**i**' value.
- If condition (**i<=10**) returns true, then again there will be one more execution of the ***statement block***. This process is repeated again and again till '**i**' becomes 11.
- When **i**=11, the condition (**i<=10**) becomes false and the loop ends and control will jump to **return 0;**, execute it and exit the program.

Additional examples for the *while* Loop

Example 1: Write a program to print sum and average of the numbers entered by user.

```
#include<stdio.h>
int main()
{
    int i=-1,num=0,sum=0;
    float avg;
    printf("\n Enter -1 to exit!");
    printf("\n Enter numbers: \n");
    while(num!=-1)
    {
        sum+=num;
        scanf("%d",&num);
        i++;
    }
    avg=(float)sum/i;
    printf("sum: %d, and average: %f\n",sum,avg);
return 0;
}
```

Example 2:Write a program to display biggest among the 5 numbers.

```
#include<stdio.h>
int main()
{
    int i=1,large=-32768, num;
    float avg;
    while(i<=5)
    {
        printf("\n Enter any number: ");
        scanf("%d", &num);
        large=large>num?large:num;
        i++;
    }
    printf("\n Biggest among 5 numbers is: %d",
large);
return 0;
}
```

Example 3: Write a program to calculate sum of numbers from m to n.

```
#include<stdio.h>
int main()
{
    int n, m, sum=0;
    printf("\n Enter -1 to exit...");
    printf("\n Enter the value of m and n: ");
    scanf("%d %d", &m, &n);
    while(m<=n)
    {
        sum+=m;
        m++;
    }
    printf("\n Sum = %d ", sum);
return 0;
}
```

Example 4:Write a program to read the numbers until -1 is encountered. Also count the number of negatives, positives and zeros entered by user.

```
#include<stdio.h>
int main()
{
    int num;
    int positives=0,negatives=0, zeros=0;
    printf("\n Enter -1 to exit...");
    printf("\n Enter any number: ");
    scanf("%d", &num);
    while(num!=-1)
    {
        if(num>0)
            positives++;
        else if(num<0)
            negatives++;
        if(num==0)
            zeros++;
        printf("\n Enter any number: ");
        scanf("%d", &num);
    }
    printf("\n No. of positive numbers: %d\tNo. of negative numbers: %d\tNo. of
zeros: %d",positives,negatives, zeros);
return 0;
}
```

Example for the nested *while* Loop

- The ***inner while loop*** defined inside the ***outer while loop*** is called as nested ***while loop***. The following is an example of how we can use a ***nested while loop,*** where we have created the 2D array (int **a[rows][columns]**) using a ***nested while loop***.
- In the example, after declaring and reading the values for **rows** and **columns**, the '**i**' is declared and it is initialized to 0. Now, the condition (***i<rows***) is tested and if it is true, then control moves to the ***inner while loop***.
- After reaching the ***inner while loop***, '**j**' is set to 0 and condition (j<columns) is tested, if it is true the value for array is read into **a[i][j]**th position in the array.

```
Syntax of nested while Loop:
    statement x;
    while (condition)
    {
        while (condition) {
        //inner loop
        statement block;
        }
        //outer loop
            statement block;
    }
    statement y;
```

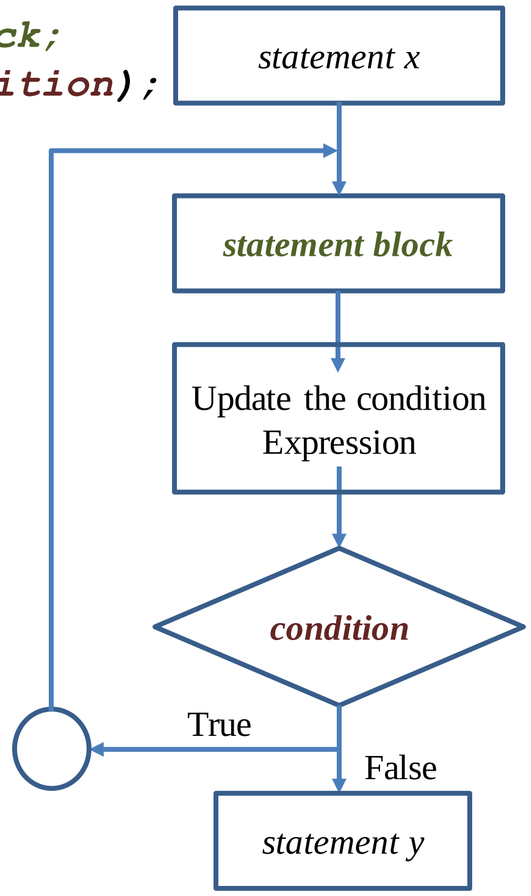
```c
#include<stdio.h>
int main()
{
   int i=0,j,rows,columns;
   printf("Enter the number of rows and columns:");
   scanf("%d %d", &rows, &columns);
   int a[rows][columns];
   while (i<rows)
   {
      j=0;
      while(j<columns)
      {
         scanf("%d",&a[i][j]);
         j++;
      }
      i++;
   }
   return 0;
}
```

Example C Program to create and read the values into 2D array.

- After reading first value into an array, now '**j**' is incremented by 1 and again condition (j<columns) is tested, if the condition is true it reads the next element into an array. Likewise values for the first row of the array are read one by one as long as the condition (j<columns) is true.
- If the condition (j<columns) is false, then the control moves to **i++;** and increases '**i**' value by 1.
- Now, the condition (***i<rows***) is tested, if the condition is true, then the ***inner while loop*** is executed one more time (that reads another row of elements). This will be repeated as long as condition (***i<rows***) is true. Otherwise control moves to **return 0,** executes it and exits the program.

3. ii) *do-while* Loop

```
Syntax of do-while Loop
   statement x;
   do
   {
   statement block;
   } while (condition);
   statement y;
```



Flowchart of the *do-while* loop.

- It is similar to ***while*** loop, but not exactly same as ***while*** loop and the only difference is that in a ***do–while*** loop, the **`condition`** is evaluated at the end of the execution of the **`statement block`**.
- Since the test **`condition`** is checked at the end, hence the body of the loop (**`statement block`**) gets executed at least once even though the **`condition`** is false.
- As it is given in the syntax, the ***do–while*** loop continues to execute while the **`condition`** is true and when the **`condition`** becomes false, the control jumps to the **`statement y.`**
- Like while, it's another indefinite loop, the number of times that loop to be executed is decided during run time.
- The major disadvantage of using a ***do–while*** loop is that it always executes at least once, so even if the user enters some invalid data, still the **`statement block`** will be executed.

Example for the ***do-while*** Loop

- For example, the following code prints the first 10 numbers using a ***do-while*** loop. In the program '**i**' is the loop variable that is initialized to 1.
- Note that before checking the condition(**i<=10**), ***statement block*** (**printf("\n %d", i);i = i + 1;**) followed by ***do*** is executed (which prints value of '**i**' and increment the '**i**' value by 1) and then condition is tested.

```
#include <stdio.h>
int main()
{
    int i = 1;
    do
    {
        printf("\n %d", i);
        i = i + 1;
        // condition updated
    } while(i<=10);
    return 0;
}
```

Example C Program to first 10 serial numbers.

- Since we are still in a ***do-while*** loop, after increasing the '**i**' value by 1, again the ***statement block*** followed by do is executed and then the condition is evaluated with the updated '**i**' value.
- If condition returns true, then again there will be one more execution of the ***statement block*** . This process is repeated again and again till '**i**' becomes 11.
- When **i**=11, the condition becomes false and the loop ends and now control jumps to **return 0**, executes it and exits the program.

Additional examples for the ***do-while*** Loop

Example 1: Write a program to list all leap years from 1900 to 2100.

```
#include<stdio.h>
int main()
{
    int  m=1900,n=2100;
    do
    {
        if(m%4==0)
        printf("\n %d is a leap year",m);
        else
        printf("\n %d is not a leap year",m);
        m++;
    }while(m<n);
return 0;
}
```

Example 2:Write a program to display square and cube of first n natural numbers.

```
#include<stdio.h>
int main()
{
    int n, i;
    printf("\n Enter the value of n: ");
    scanf("%d", &n);
    printf("\n ____________________________________________________ ");
    i=1;
    do
    {
        printf("\n \t %d \t|\t  %d \t|\t  %ld\t|",i*i,i*i*i);
        i++;
    }while(i<n);
    printf("\n ____________________________________________________ ");
return 0;
}
```

Example 3: Write a program to read a character until a * is encountered. Also count number of uppercase, lowercase and numbers entered.

```
#include<stdio.h>
int main()
{
   int lowers=0, uppers=0,numbers=0;
   char ch;
   printf("\n Enter any character: ");
   scanf("%c", &ch);
   do
   {
      if (ch>='A' && ch<='Z')
         uppers++;
      else if (ch>='a' && ch<='z')
         lowers++;
      else
         numbers++;
      scanf("%c", &ch);
   }while(ch!='*');
   printf("\n Uppercase letters = %d \t Lowercase letters = %d \t
Numbers = %d",uppers, lowers, numbers-uppers);
return 0;
}
```

Example 4:Write a program to read numbers until -1 is encountered. Also sum and mean of all the positive numbers and negative numbers.

```
#include<stdio.h>
int main()
{
   int num;
   int sum_positive=0,sum_negative=0, positives=0,negatives=0;
   float pos_mean,neg_mean;
   printf("\n Enter -1 to exit...");
   printf("\n Enter any number: ");
   scanf("%d", &num);
   do
     {
     if(num>0){
        sum_positive+=num;
        positives++;
     }
     else if(num<0){
        sum_negative+=num;
        negatives++;
     }
     printf("\n Enter any number: ");
     scanf("%d", &num);
   }while(num!=-1);
   pos_mean=(float)sum_positive/positives;
   neg_mean=(float)sum_negative/negatives;
   printf("\nSum of +ve numbers = %d\t Sum of -ve numbers = %d\t +ve mean = %f\t -ve
mean = %f",sum_positive, sum_negative,pos_mean,neg_mean);
return 0;
}
```

Example for the nested *do-while* Loop

```
Syntax of do-while Loop
   statement x;
   do
   {
      do
      {
      //inner do-while
   statement block;
     } while (condition);
      //inner do-while
   statement block;
} while (condition);
statement y;
```

```
#include<stdio.h>
int main()
{
   int i=1;
   do {
      int j=1;
      do
      {
         printf("*");
         j++;
      }while(j<=4);
      printf("\n");
      i++;
   }while(i<=4);
   return 0;
}
```

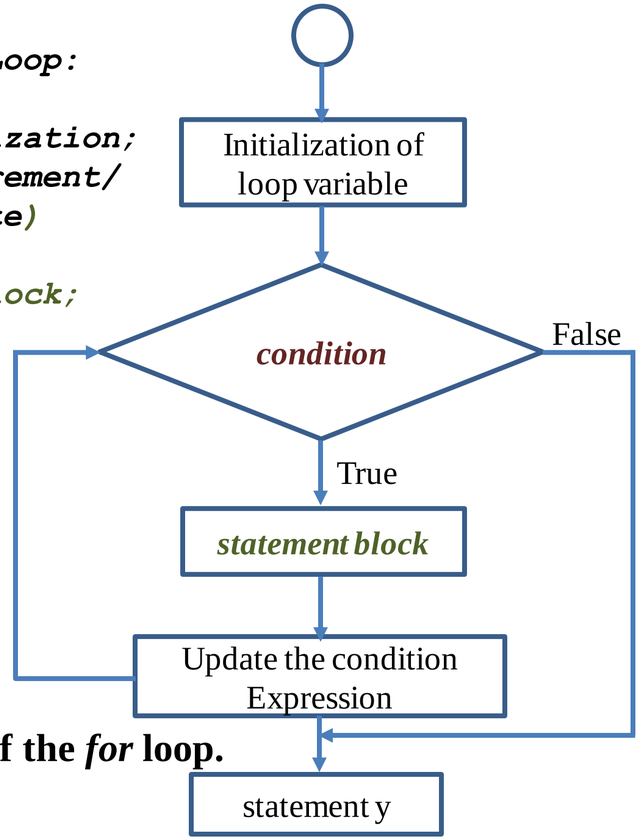
- Use of ***do-while*** inside another loop structure is called as nested ***do-while*** loop.
- In the program, we initialize the outer loop counter/loop variable, i.e., i=1. Since ***do-while*** is an exit control loop, so the inner ***do-while*** will be executed without checking the condition(**i<=4**) in the outer ***do-while*** loop.
- During the execution of the inner loop, '*' is printed and loop variable of the inner loop is incremented by 1 (i.e., **j++**), and the condition of inner loop is tested (i.e., **j<=8** is tested).
- If the condition returns true, then again there will be one more execution of the statement block of the inner loop. This process is repeated again and again till '**j**' becomes 9.
- Then control jumps to **printf("\n");** statement in the outer loop and executes it (which prints a new line), now the outer loop counter is increased by 1 (i.e., **i++**) and the condition in the outer loop is tested.
- If the condition of the outer loop returns true, then again there will be one more execution of the inner loop. This process is repeated again and again till '**i**' becomes 5.
- Then the control is moved to **return 0**, executes it and exits the program.

Example C Program to print rectangular pattern of stars (4X8).

3. iii) *for* Loop

- Similar to the ***while*** and ***do–while*** loops, the ***for*** loop provides a mechanism to repeat a task until a particular ***condition*** is true.
- Being a definite loop, the ***for*** loop is used to repeat execution of a ***statement block*** for a fixed number of times.
- As per syntax of ***for*** loop, it has three sections in it i.e., ***initialization***, ***condition*** and ***increment/ decrement*** and all of these must be separated by semicolon.

```
Syntax of for Loop:
   statement x;
   for(initialization;
condition; increment/
decrement/update)
   {
   statement block;
   }
   statement y;
```



Flowchart of the *for* loop.

- Multiple initialisations needs to be separated by comma.
- If the ***condition*** section is empty, then it assumes the ***condition*** is true and it becomes an infinite loop. Multiple test conditions can be tested using logical operators.
- A loop variable is initialized once and tested before execution of the ***statement block***. Never use floating point variable as loop control variable.
- For every iteration loop variable is updated either by incrementing or decrementing value of the loop variable in the for statement or within the statement block and the ***condition*** is then tested.

Example of the *for* Loop


```
Syntax of for loop
   statement x;
   for(initialization; condition;
increment/decrement/update)
   {
   statement block;
   }
   statement y;
```

```
#include <stdio.h>
int main()
{
   int i;
   for(i=0;i<=10;i++)
   {
      printf("\n %d", i);
   }
return 0;
}
```

Condition

Initialization

increment/decrement/update

Statement block

Statement y

Example C Program to print first 10 serial numbers using for loop

- According to the syntax of ***for*** statement, if the ***condition*** in the loop becomes true, ***statement block*** is executed, otherwise the execution of the ***statement block*** of the ***for*** loop is skipped and the control jumps to ***statement y***.
- In the program, loop variable '**i**' is declared as an int type.
- After specifying the initial value of '**i**', ***condition***, and ***increment*** of '**i**'. the value of the '**i**' is evaluated using relational expression '**i<10**'.
- Based on the outcome of **'i<10'**, control is shifted to either ***statement block*** or ***statement y***.
- If an outcome of the condition is true, then the control is shifted to ***statement block*** otherwise it is shifted to ***statement y***.
- As long as an outcome of '**i<10**' is true, every updated value of '**i**' is printed. When the outcome of '**i<10**' is false, control is shifted from loop to ***statement y***.

Additional examples of the *for* Loop

Example 1: Write a program to calculate average of first n numbers.

```
#include<stdio.h>
int main()
{
    int n, sum=0, count=0;
    float avg;
    printf("\n Enter the value of n: ");
    scanf("%d", &n);
    for (i=1;i<n;i++)
    {
        sum=+i;
    }
    avg=(float)sum/count;
    printf("\n Sum = %d \t average = %f", sum, avg);
return 0;
}
```

Example 2:Write a program to print the following patter.

Pass 1 - 1 2 3 4 5
Pass 2 - 1 2 3 4 5
Pass 3 - 1 2 3 4 5
Pass 4 - 1 2 3 4 5
Pass 5 - 1 2 3 4 5

```
#include<stdio.h>
int main()
{
    int i, j;
    for (i=1;i<=5;i++)
    {
        printf("\n Pass %d - ",i );
        for(j=1;j<=5;j++)
            printf(" %d", j);
    }
return 0;
}
```

Example 3: Write a program to print the following patter.

```
    1
   12
  123
 1234
12345
```

```
#include<stdio.h>
#define N 5
int main()
{
    int i, j, k;
    for (i=1;i<=N;i++)
    {
        for(k=N;k>=i;k--)
            printf(" ");
            for(j=1;j<=i;j++)
                printf("%d",j);
                printf("\n");
    }
return 0;
}
```

Example 4: Write a program to print the following Pascal Triangle.

```
      1
     121
    12321
   1234321
  123454321
```

```
#include<stdio.h>
#define N 5
int main()
{
    int i, j, k, l;
    for (i=1;i<=N;i++)
    {
        for(k=N;k>=i;k--)
            printf(" ");
        for(j=1;j<=i;j++)
            printf("%d",j);
        for(l=j-2;l>0;l--)
            printf("%d",l);
        printf("\n");
    }
return 0;
}
```

Example of the nested *for* Loop

- Use of ***inner for loop*** in an ***outer for loop*** is called as ***nested for loop*** and any number of ***for loops*** can be defined inside another loop. The following program display n multiplication tables.

```
Syntax of nested for loop:
for (initialization; condition; update)

{
    for(initialization; condition; update
)
    {
            // inner loop statements.
    }
    // outer loop statements.
}
```

```c
#include<stdio.h>
int main()
{
   int i,j,n;
   printf("Enter the value of n: ");
   scanf("%d",&n);
   for (i=1;i<=n;i++)
   {
      for(j=1;j<=10;j++)
      {
         printf("%d\t",(i*j));
      }
      printf("\n");
   }
   return 0;
}
```

Example C Program to display 1 to n multiplication tables.

- In the outer loop, the '**i**' variable is initialized to 1 and then the condition is tested (i.e., **i<=n**).
- If the condition is true, then the program control passes to the inner loop.
- After the execution of the inner loop, the control moves back to increment the outer loop variable, i.e., i++, and condition (**i<=n**) in the outer loop is tested.
- If the condition in outer loop is true, then the inner loop will be executed again. Inner loop gets executed as long as the condition in the outer loop is true.
- Once condition in outer becomes false, then control passed to the next section of code followed by ***nested for*** statement.

Infinite Loops

- Loop that executes forever is called as an infinite loop, which produces continuous output or no output. They are useful for applications that accept the user input and generate the output continuously until the user exits from the application manually.
- In the following situations, this type of loop can be used:
  1. All the operating systems run in an infinite loop,
  2. All the servers run in an infinite
  3. loop, All the games etc.,
  4. Some of the real time scenarios, where infinite loops exist.
- We can create an infinite loop through various control structures. The following are the control structures through which we will define the infinite loop:
  - for loop
  - while loop
  - do-while loop
  - go to statement

Infinite Loops….contd.,

control statement	***for***	***while***	***do-while***	***goto***

Syntax	for(; ;)
	{
	// body of the for loop.
	}
		while(1)
		{
		// body of the loop..
		}
			do
			{
			// body of the loop..
			}while(1);
				label;
				// body statements.
				goto label;

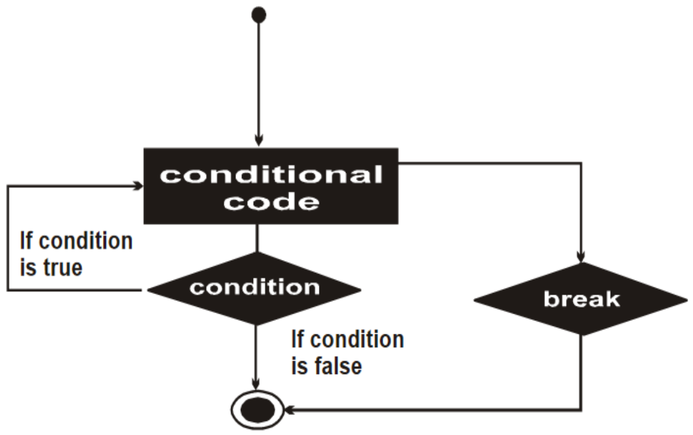
Example	#include<stdio.h>
	int main()
	{
	for(;;)
	{
	printf(“Hello RVCE”);
	}
		#include<stdio.h>
		int main()
		{
		while(1)
		{
		printf(“Hello RVCE”);
		}
			#include<stdio.h>
			int main()
			{
			do
			{
			printf(“Hello RVCE”);
			}while(1);
				#include<stdio.h>
				int main()
				{
				MyLabel:
				printf("Hello RVCE");
				goto MyLabel;
				}

	In the above example for loop is not defined completely, all sections are empty. When condition is empty, it is assumed true by compiler. That is why it becomes infinite loop	For while and do-while statement, in place of test expression (condition) in while loop, if we put 1, while condition is true. This is because any non-zero value is considered as true by a while loop.		In the above program MyLabel is label created just before “printf(“Hello RVCE”); so after executing “printf(“Hello RVCE”);” statement, control goes to goto line, meanwhile we know that goto statement passes control to the position in the program, where corresponding label is created.

Loop control statements

- Loop control statements are those statements which regulate the way loop structure works, such that they can prematurely terminate the loop in between, they can even skip the execution of a certain iteration in the loop and even they can shift control from the loop to any other part of the program.
- These statements change execution of the statement block from its normal sequence. There are basically three loop control statements viz., **break, continue and goto**.

break Statement



Flowchart of the *break*.

- ***break*** is used to terminate the loop prematurely in which it appears.
- We know so far its usage with switch-case construct and its also widely used with for, while and do-while constructs.
- When the compiler encounters a ***break*** statement, the control passes to the statement that follows the loop in which the break statement appears.
- syntax is quite simple, just type keyword ***break*** followed by a semi-colon.

  break;

Loop control statements….contd.,

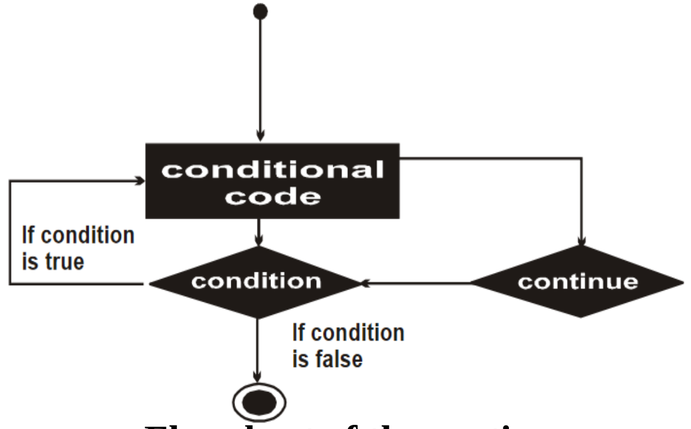
- The following program given below shows how one can use a break statement to terminate the loop in which it is appears.
- As soon as '**i**' becomes equal to 5, the break statement is executed and the control jumps to the statement following the while loop (i.e., return 0). Hence, the break statement is used to exit a loop from any point within its body, bypassing its normal termination expression.

```
#include <stdio.h>
int main(){
int i = 0;
while(i<=10) {
if (i==5)
break;
printf("\t %d", i);
i = i + 1;
}
return 0;
}
```

continue Statement

- It is used to skip the iteration in the loop based on the condition.
- like the break statement, the continue statement cannot be used without an enclosing ***for***, ***while***, or ***do–while*** loop.
- When the compiler sees a ***continue*** statement, the statements after the ***continue*** statement are skipped and then control is unconditionally transferred to code that tests the controlling expression.

Loop control statements….contd.,



Flowchart of the *continue*.

syntax

just type keyword ***continue*** followed by a semi-colon.

continue;

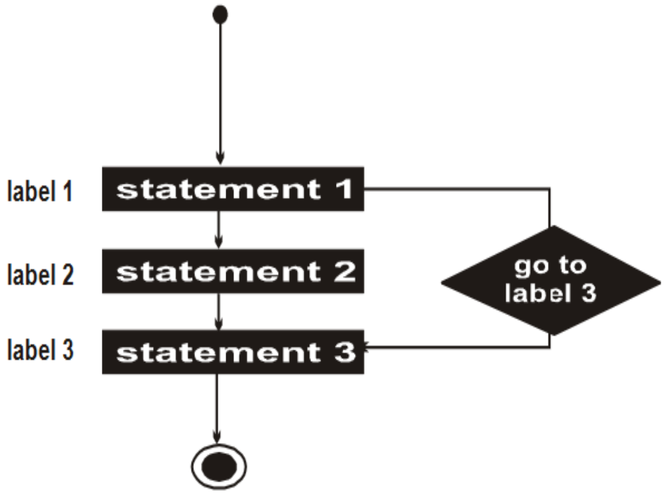
If the continue statement is used in a for loop, control is transferred to an expression that updates the loop variable.

Example

```
#include <stdio.h>
int main()
{
int i;
for(i=0; i<= 10; i++)
{
if (i==5)
continue;
printf("\t %d", i);
}
return 0;
}
```

- Note that the code is meant to print numbers from 0 to 10. But the loop prints all numbers from 0 to 10 but skip printing of the number 5.
- In the following code when loop variable '**i**' becomes 5, if condition(i==5) returns true and passes control to ***continue*** statement.
- When the compiler encounters a ***continue*** statement, the printf() statement is skipped and the control passes to the expression that increments the value of 'i' (i.e., i++).
- Unlike break, ***continue*** make control to stay with the loop by skipping specific iteration in the loop.

Loop control statements….contd.,



Flowchart of the *goto*.

- ***goto* statement:** It is used for unconditional transfer of control from goto statement to a labeled statement in the program. Before using goto statement one needs to label statement or statement block to which he/she wants to transfer control from goto statement.
- Use of goto statement is highly discouraged in any programming language because it makes it difficult to trace the flow of control in a program, making the program hard to understand and hard to modify.

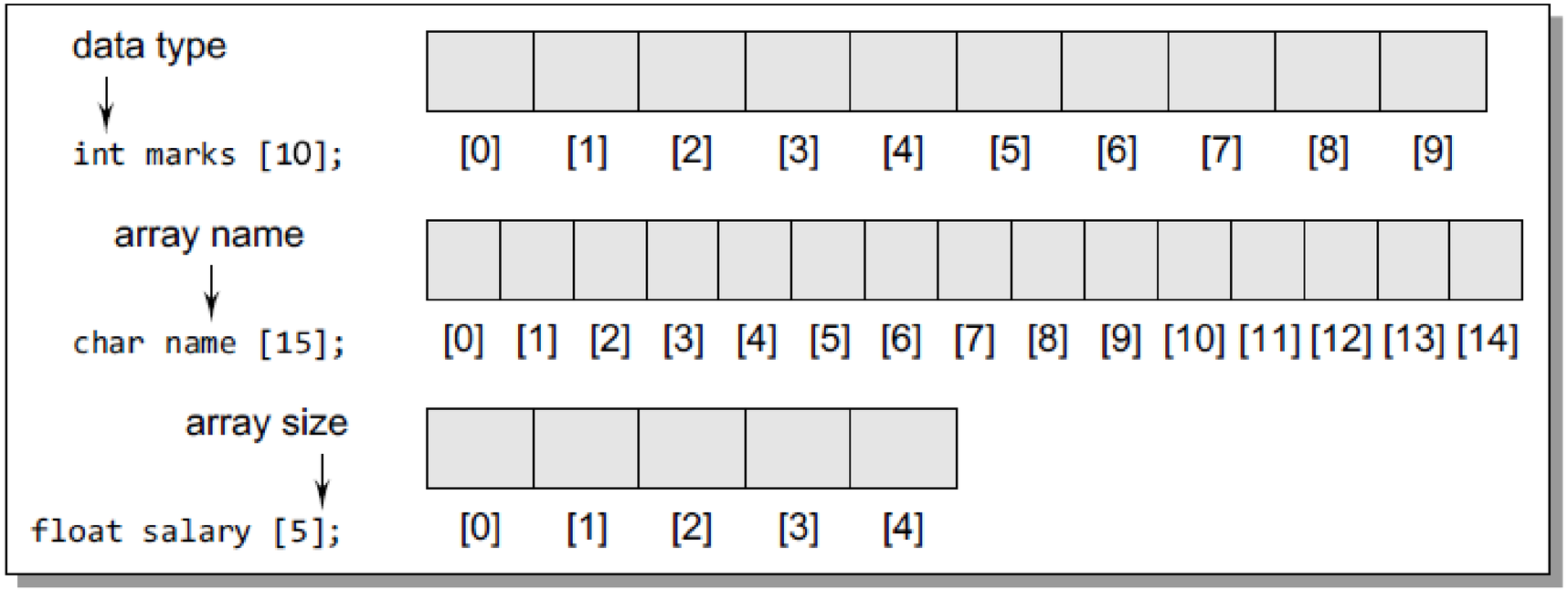
- In the following example, the for loop is supposed to print numbers from 0 to 10, but when the counter variable '**i**' becomes 5, if statement evaluates the condition (i==5).

```
#include <stdio.h>
int main()
{
int i;
char choice;
repeat1to4:
for(i=0; i<= 10; i++)
{
if (i==5){
printf("Do you want print 1-4 again [y/n]: ?");
scanf("%c",&choice);
If(choice=='y')
goto repeat1to4;
else
exit(0);
}
printf("\t %d", i);
}
return 0;
}
```

- If condition is true, then the control passed to ***printf*** statement and reads user choice value, and now inner ***if*** condition (choice=='y') is evaluated. Otherwise control passed to exit(0), that makes the program to exit.
- If the outcome of the condition (choice=='y') is true, now control is passed to goto statement, which passes control to label ***repeat1to4*** just before the ***for*** loop and ***for*** loop restarts again.

Declaration of Arrays

- An array is a collection of similar data elements and they all have the same data type.
- The elements of the array are stored in consecutive memory locations and are referenced by an index (also known as the subscript).
- The array index is an ordinal number which is used to identify an element of the array.
- Declaring an array means specifying the following:
    1) Data type—the kind of values it can store, for example, int, char, float, double.
    2) Name—to identify the array.
    3) Size—the maximum number of values that the array can hold.
- Arrays are declared using the syntax: type name[size];
- For example: int marks[10];



Declaring arrays of different data types and sizes

- In C, the array index starts from zero. The first element will be stored in marks[0], second element in marks[1], and so on. Therefore, the last element, that is the 10th element, will be stored in marks[9].

Accessing Elements of an Array

- Storing related data items in a single array enables the programmers to develop concise and efficient programs. But there is no single function that can operate on all the elements of an array.
- To access all the elements, we must use a loop, i.e., we can access all the elements of an array by varying the value of the array subscript or index. But note that the subscript must be an integral value or an expression that evaluates to an integral value.

```
// Set each element of the array to -1
int i, marks[10];
for(i=0;i<10;i++)
        marks[i] = -1;
```

Code to initialize each element of the array to –1

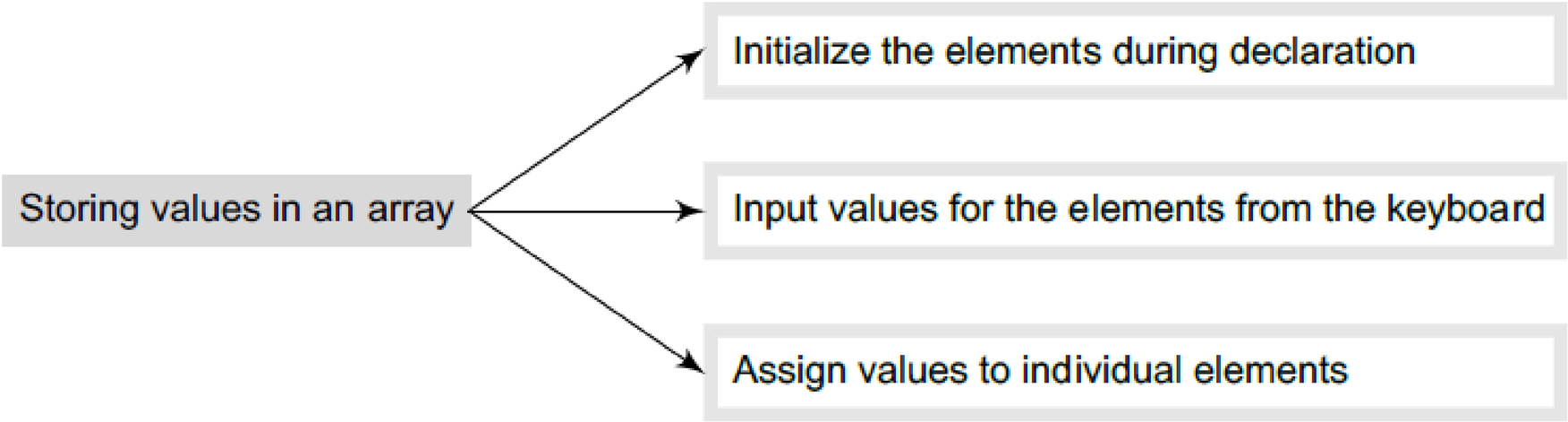
Address of Array Elements

- Since an array stores all its data elements in consecutive memory locations, storing just the base address, that is the address of the first element in the array, is sufficient. The address of other data elements can simply be calculated using the base address.
- The formula to perform this calculation is,

$$\text{Address of data element at index } k = BA(A) + w(k - \text{lower_bound})$$

- Here, A is the array, k is the index of the element of which we have to calculate the address, BA is the base address of the array A, and w is the size of one element in memory.
- The length of an array is given by the number of elements stored in it.

Storing values in Arrays



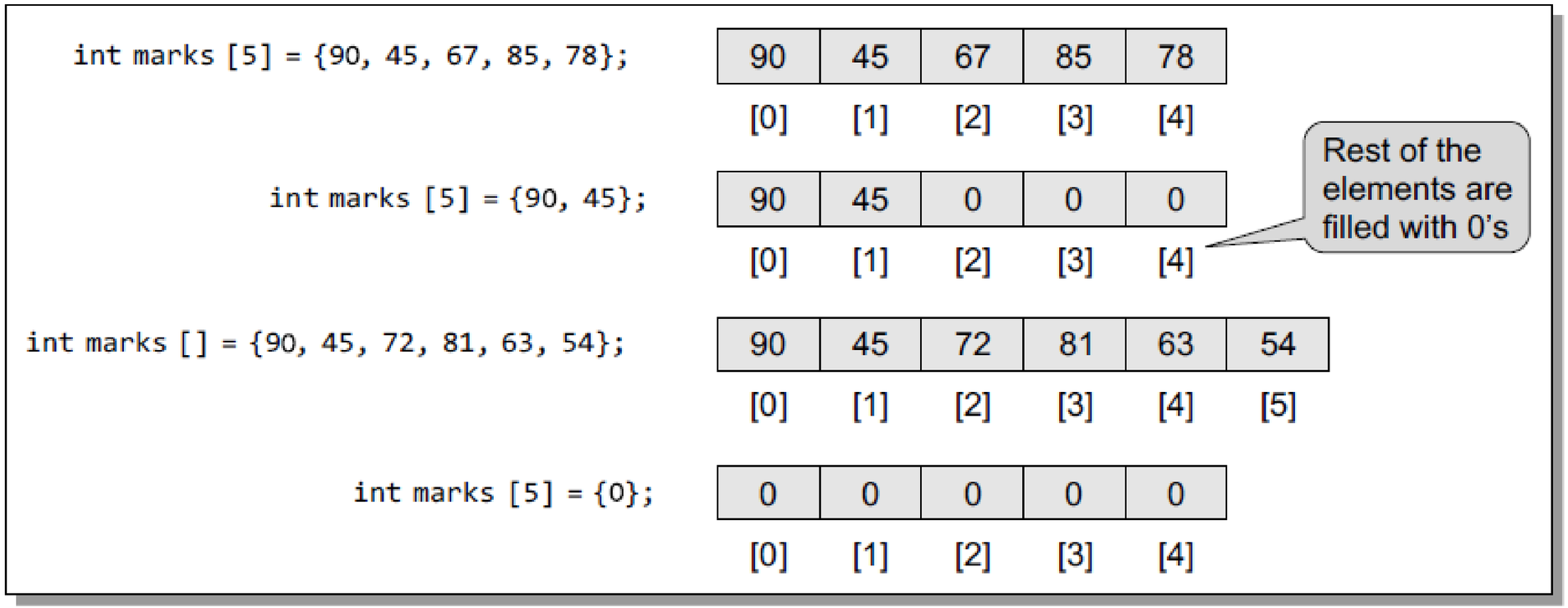
Inputting Values for Array Elements

- In this method, a while/do–while or a for loop is executed to input the value for each array element. An example code is given below.

```
int i, marks[10];
for(i=0;i<10;i++)
        scanf("%d", &marks[i]);
```

Initializing Array Elements during Declaration

- The elements of an array can be initialized at the time of declaration, just like any other variable. Various ways of initializing array elements is given below.



Initialization of array elements

Assigning Values to Individual Elements

- In this method, individual elements of the array are given values by using the assignment operator. For example: marks[3] = 100;
- It should be noted that we cannot assign one array to another array, even if the two arrays have the same type and size. To copy an array, the value of every element of the first array must be copied into the respective elements of the second array.

```
int i, arr1[10], arr2[10];
arr1[10] = {0,1,2,3,4,5,6,7,8,9};
for(i=0;i<10;i++)
        arr2[i] = arr1[i];
```

Code to copy an array at the individual element level

Operations On Arrays

There are a number of operations that can be preformed on arrays and they are as follows.

- Traversing an array
- Inserting and deleting an element in an array
- Searching an element in an array
- Merging two arrays
- Sorting an array in ascending or descending order

Traversing an array:

- Traversing the data elements of an array can include printing every element, counting the total number of elements, or performing any process on particular elements. Since, array is a linear data structure, traversing its elements is very simple and straightforward.

Program to read and display *n* numbers using an array

```
Step 1: [INITIALIZATION] SET I = lower_bound
Step 2: Repeat Steps 3 to 4 while I <= upper_bound
Step 3:      Apply Process to A[I]
Step 4:      SET  I = I + 1
        [END OF LOOP]
Step 5: EXIT
```

Algorithm for array traversal

```
#include <stdio.h>
#include <conio.h>
int main()
{
int i, n, arr[20];
printf("\n Enter the number of elements in the array : ");
scanf("%d", &n);
for(i=0; i<n; i++)
{
printf("\n arr[%d] = ", i);
scanf("%d",&arr[i]);
}
printf("\n The array elements are ");
for(i=0; i<n; i++)
printf("\t %d", arr[i]);
return 0; }
```

Program to print the position of the smallest number in an array of *n* numbers.

```
#include <stdio.h>
#include <conio.h>
int main()
{
int i, n, arr[20], small, pos;
printf("\n Enter the number of elements
     in the array : ");
scanf("%d", &n);
printf("\n Enter the elements : ");
for(i=0; i<n; i++)
scanf("%d",&arr[i]);
small = arr[0];
pos =0;
for(i=1; i<n; i++)
{
  if(arr[i]<small)
  {
  small = arr[i];
  pos = i;
  }
}
printf("\n The smallest element is : %d",
     small);
printf("\n The position of the smallest
     element in the array is : %d", pos);
return 0;
}
```

Program to find whether an array of integers contains a duplicate number.

```
#include <stdio.h>
#include <conio.h>
int main()
{
int array[10], i, n, j, flag =0;
printf("\n Enter the size of the array : ");
scanf("%d", &n);
for(i=0; i<n; i++)
{
printf("\n array[%d] = ", i);
scanf("%d", &array[i]);
}
for(i=0; i<n; i++)
{
   for(j=i+1; j<n; j++)
  {
   if(array[i] == array[j] && i!=j)
  {
  flag =1;
  printf("\n Duplicate numbers found at
     locations %d and %d", i, j);
  }
  }
}
if(flag == 0)
printf("\n No Duplicates Found");
return 0;
}
```

Inserting an Element in an Array

- If an element has to be inserted at the end of an existing array, then the task of insertion is quite simple. We just have to add 1 to the upper_bound and assign the value.
- Here, we assume that the memory space allocated for the array is still available. For example, if an array is declared to contain 10 elements, but currently it has only 8 elements, then obviously there is space to accommodate two more elements. But if it already has 10 elements, then we will not be able to add another element to it.

```
Step 1: Set upper_bound = upper_bound + 1
Step 2: Set A[upper_bound] = VAL
Step 3: EXIT
```

Algorithm to append a new element to an existing array

Inserting an Element in an Array

- If an element has to be inserted in the middle of the array, then the task of insertion involves the algorithm shown here.

- To do this process, the location where the new element will be inserted is to be found out.
- Then all the elements to the right of this array location has to be moved one position right, so that space can be created to store the new value.

```
Step 1: [INITIALIZATION] SET I = N
Step 2: Repeat Steps 3 and 4 while I >= POS
Step 3:          SET A[I + 1] = A[I]
Step 4:          SET I = I - 1
        [END OF LOOP]
Step 5: SET N = N + 1
Step 6: SET A[POS] = VAL
Step 7: EXIT
```

Algorithm to insert an element in the middle of an array.

Deleting an Element in an Array

- Deleting an element from an array means removing a data element from an already existing array. Deleting an element at the end of the existing array is quite simple.

- To delete an element in the middle of an array, we must first find the location from where the element has to be deleted and then move all the elements one position towards left.
- Thus the space vacated by the deleted element will be occupied by rest of the elements.

```
Step 1: SET upper_bound = upper_bound - 1
Step 2: EXIT
```

Algorithm to delete the last element of an array

```
Step 1: [INITIALIZATION] SET I = POS
Step 2: Repeat Steps 3 and 4 while I <= N - 1
Step 3:          SET A[I] = A[I + 1]
Step 4:          SET I = I + 1
        [END OF LOOP]
Step 5: SET N = N - 1
Step 6: EXIT
```

Algorithm to delete an element from the middle of an array

Two Dimensional Arrays

- A two-dimensional array is declared as: data_type array_name[row_size][column_size];
- Therefore, a two-dimensional $m \times n$ array contains m*n data elements and each element is accessed using two subscripts, i and j, where i <= m and j <= n.

Rows / Columns	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	marks[0][0]	marks[0][1]	marks[0][2]	marks[0][3]	marks[0][4]
Row 1	marks[1][0]	marks[1][1]	marks[1][2]	marks[1][3]	marks[1][4]
Row 2	marks[2][0]	marks[2][1]	marks[2][2]	marks[2][3]	marks[2][4]

Two-dimensional array

- A two-dimensional array is initialized in the same way as a one-dimensional array is initialized. For example: int marks[2][3]={90, 87, 78, 68, 62, 71};
- Note that the initialization of a two-dimensional array is done row by row. The above statement can also be written as: int marks[2][3]={{90,87,78},{68, 62, 71}};

Two Dimensional Arrays

- In case of one-dimensional arrays, we used a single *for* loop to vary the index *i* in every pass, so that all the elements could be scanned. Since the two-dimensional array contains two subscripts, we will use two *for* loops to scan the elements.
- The first *for* loop will scan each row in the 2D array and the second *for* loop will scan individual columns for every row in the array.

Program to print the elements of a 2D array

```
#include <stdio.h>
#include <conio.h>
int main()
{
int arr[2][2] = {12, 34, 56,32};
int i, j;
for(i=0;i<2;i++)
{
printf("\n");
for(j=0;j<2;j++)
printf("%d\t", arr[i][j]);
}
return 0;
}
```

Operations on Two Dimensional Arrays

- Two-dimensional arrays can be used to implement the mathematical concept of matrices. In mathematics, a matrix is a grid of numbers, arranged in rows and columns. Thus, using two dimensional arrays, we can perform the following operations on an $m \times n$ matrix:

1) **Transpose** of an $m \times n$ matrix A is given as a $n \times m$ matrix B, where $B_{i,j} = A_{j,i}$.
2) **Sum and Difference:** Two matrices that are compatible with each other can be added or subtracted together, storing the result in the third matrix. Two matrices are said to be compatible when they have the same number of rows and columns. The elements of two matrices can be added by writing:

$$C_{i,j} = A_{i,j} + B_{i,j}$$

The elements of two matrices can be subtracted by writing:

$$C_{i,j} = A_{i,j} - B_{i,j}.$$

Operations on Two Dimensional Arrays

3) **Product:** Two matrices can be multiplied with each other if the number of columns in the first matrix is equal to the number of rows in the second matrix. Therefore, $m \times n$ matrix A can be multiplied with a $p \times q$ matrix B if $n = p$. The dimension of the product matrix is $m \times q$. The elements of two matrices can be multiplied by writing:

$$C_{i,j} = \sum A_{i,k} B_{k,j} \qquad \text{for k = 1 to n}$$